

	담당	원장
결재		

자바 스프링 리액트로 완성하는 클라우드 활용 풀스택 개발

3차 프로젝트 완료 보고서

웹 기반 실시간 합주 및 음악 협업 종합 플랫폼

2025.01.28

구성원 : 신지웅
전유진
홍채원

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

문서 개정 이력

개정번호	개정일자	시행일자	개정내용	담당자
1.0	2026.01.24	2026.01.24	최초 작성	홍채원
1.1	2026.01.25	2026.01.25	내용 1차 수정	홍채원
2.0	2026.01.26	2026.01.26	첨부 파일 추가	홍채원
2.1	2026.01.26	2026.01.26	내용 2차 수정	홍채원
2.2	2026.01.27	2026.01.27	내용 3차 수정	홍채원
3.0	2026.01.28	2026.01.28	첨부 파일 수정 및 작성 완료	홍채원

교 육 기 관 : 한 국 정 보 교 육 원
팀 명 : 예 외 처 리 반
팀 장 : 신 지 응
팀 원 : 홍 채 원
전 유 진

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

목차

1. 프로젝트 개요	
● 프로젝트 명	4
● 프로젝트 기간	4
● 프로젝트 목표	4
● 프로젝트 시나리오	4
● 프로젝트 수행 요건	4
2. 프로젝트 추진 체계	
● 프로젝트 참여 인력 총괄표	5
● 참여 인력 업무 분장	5
3. 세부 프로젝트 내용	
● 전체 구성도	6
● 네트워크 구성도	9
● 서버(물리/가상) 구성 현황	9
● 네트워크 구성 현황	10
● 상세 구축 및 구성 내용	11
● 구축 결과	32
4. 개별 후기	34

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

1. 프로젝트 개요

- 프로젝트 명
웹 기반 실시간 합주 및 음악 협업 종합 플랫폼
- 프로젝트 기간
2025.12~2025.12
- 프로젝트 목표
주요 목표: 웹 기반 실시간 저지연 합주 및 음악 협업 종합 플랫폼 구축
세부 목표:
 - 실시간 합주 시스템: WebRTC P2P 기술, 저지연 목표 구현
 - AI 음원 분리: Demucs 기반 4트랙 분리, MR 제작, 연습 도구 개발
 - 커뮤니티: 게시판, 실시간 댓글, 협업 파트너 모집, DM
 - 마켓플레이스: 중고 악기 거래, 검색, 판매자 프로필
 - 요금제: 3단계 Freemium, 토크페이먼트 연동
 - 웹 접근성: SPA 구조, 크로스 플랫폼, 반응형 디자인
- 프로젝트 시나리오
 1. 일반 사용자 (연주자)

일반 사용자



2. 관람자

관람자



3. 합주실 호스트

합주실 호스트



4. 프리미엄 사용자(구현 예정)

프리미엄 사용자(구현 예정)



- 프로젝트 수행요건
 - 기술 요건: React, TypeScript, Supabase, WebRTC, 토크페이먼트 등
 - 성능 요건: 레이턴시 50-120ms, 최대 8명 동시 접속 등 구체적 수치
 - 보안 요건: JWT, RLS, HTTPS/WSS, PCI-DSS 준수
 - 호환성 요건: Chrome/Firefox/Safari/Edge 90+ 지원

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

2. 프로젝트 추진 체계

- 프로젝트 참여 인력 총괄표

성명	소속	역할	담당업무
신지웅	대진대학교	Project Leader	실시간 오디오통신 개발 및 프로젝트 총괄
전유진	대진대학교	Project Assistant	프론트/백엔드 개발 및 결제 시스템 구축
홍채원	대진대학교	Project Assistant	보안 인증 및 부가 서비스 개발, 문서화

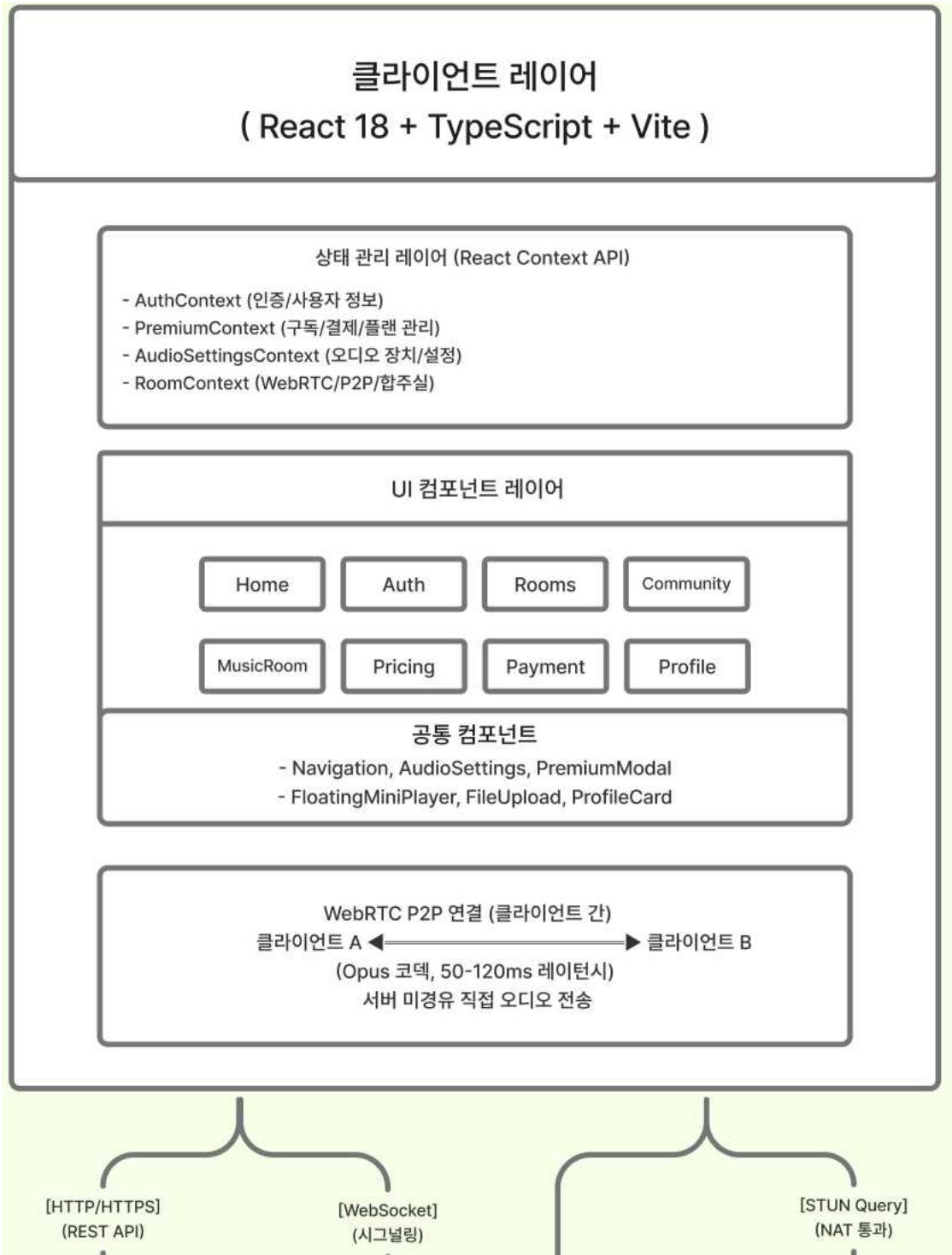
- 참여 인력 업무 분장

업무명	업무내용
실시간 오디오통신 개발 및 프로젝트 총괄	- WebRTC P2P 오디오 전송 - WebSocket 시그널링 서버 - Web Audio API (볼륨, 패닝) - 성능 최적화 및 프로젝트 총괄
프론트/백엔드 개발 및 결제 시스템 구축	- Supabase 데이터베이스 설계 - UI/UX 디자인 및 React 컴포넌트 - Express REST API 서버 - 결제 시스템 (토스페이먼츠)
보안 인증 및 부가 서비스 개발, 문서화	- RLS 보안 정책 및 인증 - 커뮤니티 게시판 - 음원 분리 서비스 (Demucs) - 문서 작성

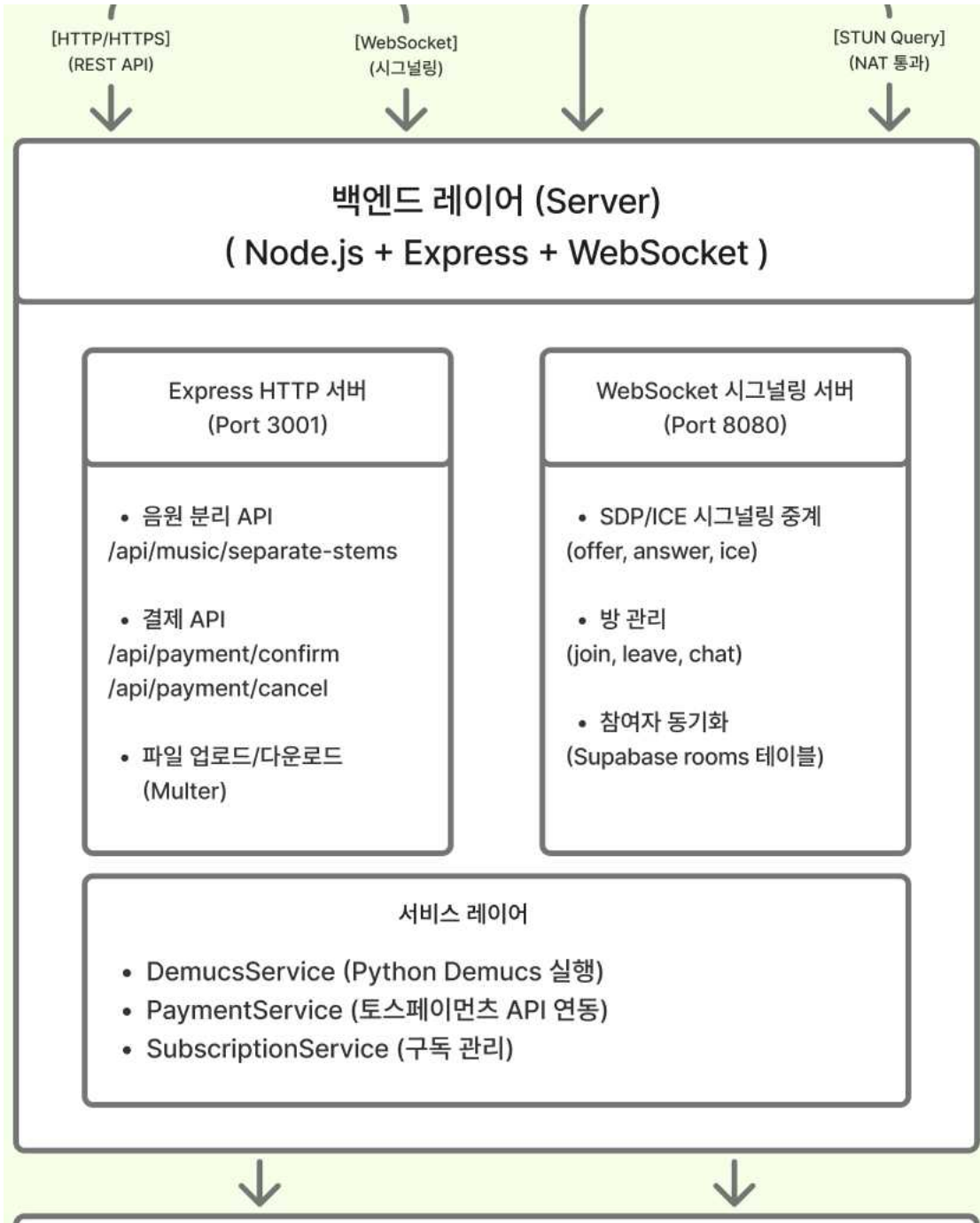
프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

3. 세부 프로젝트 내용

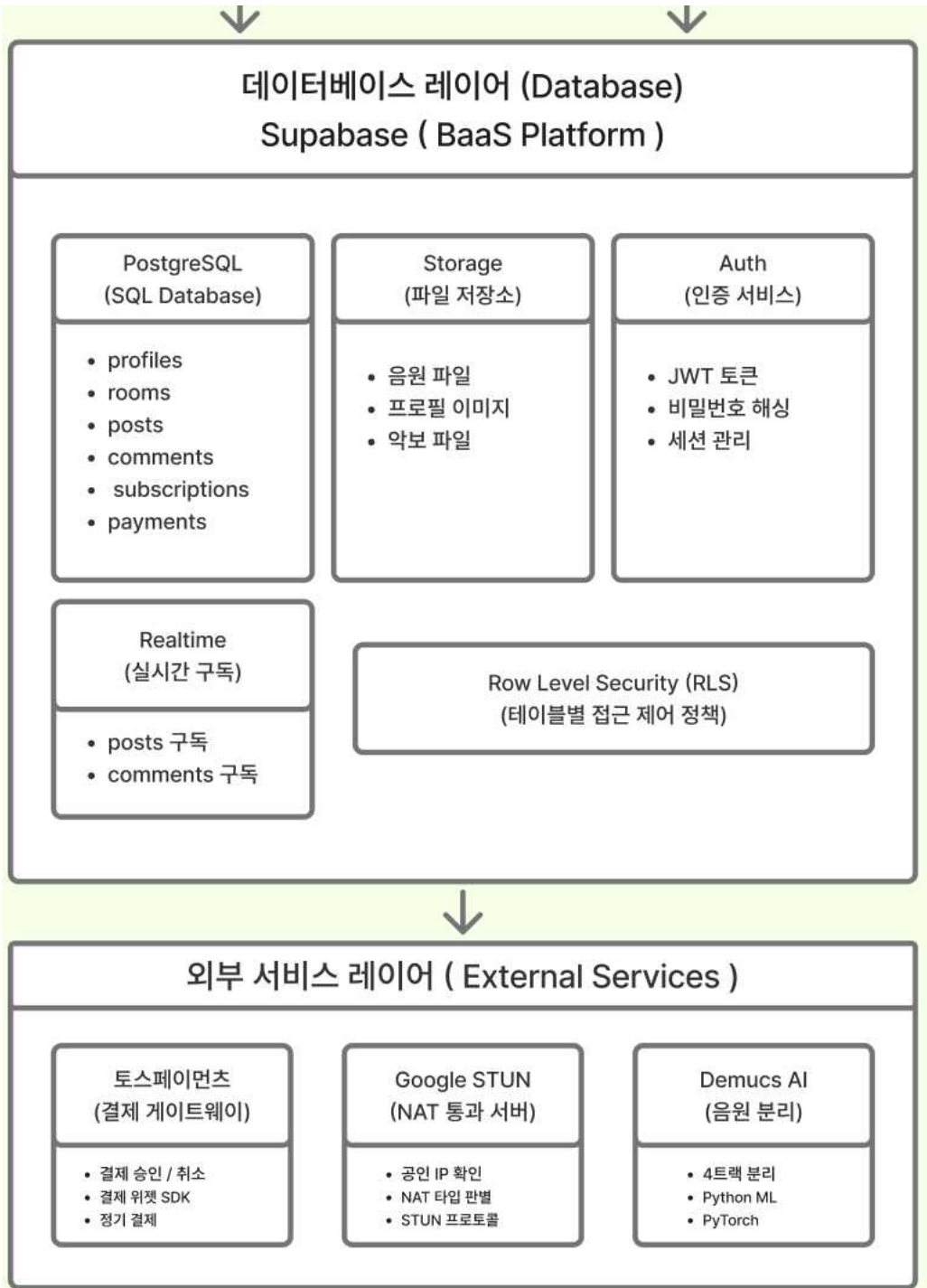
3.1 전체 구성도



프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

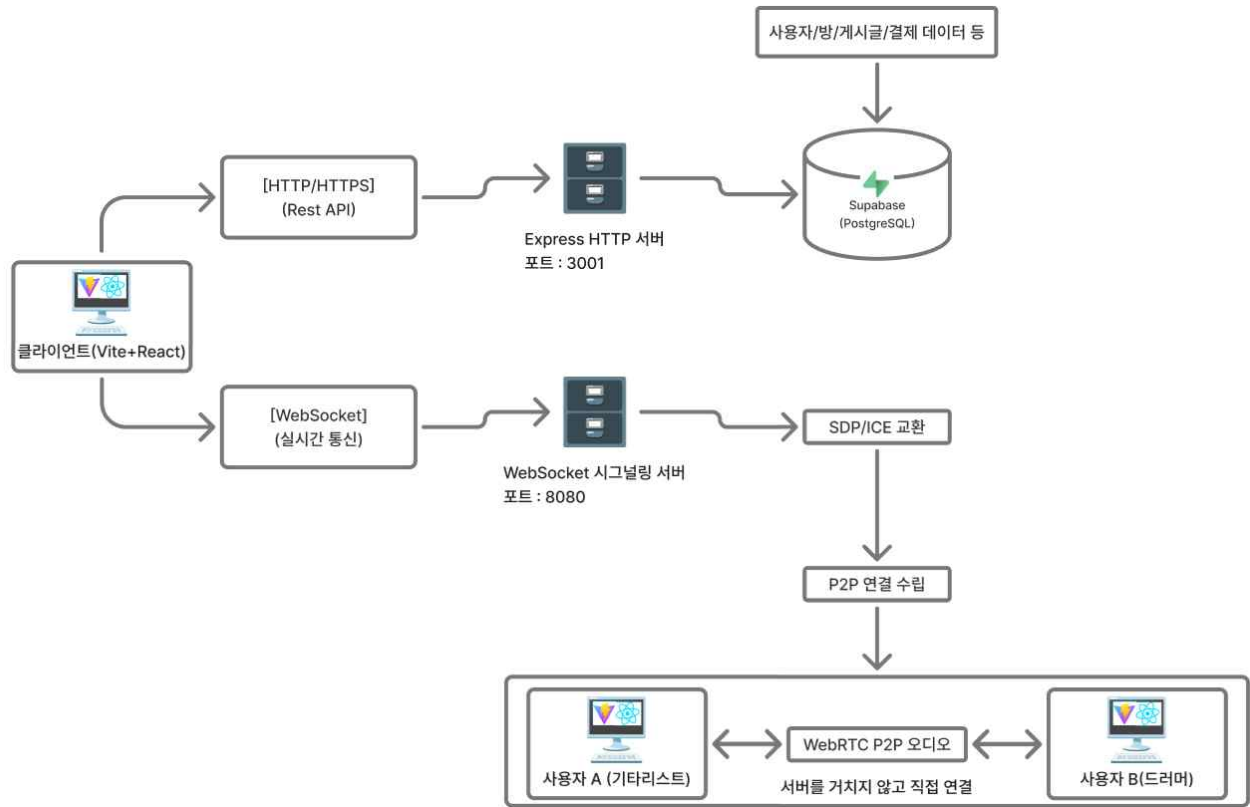


프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28



프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

3.2 네트워크 구성도



3.3 서버(물리/가상) 구성 현황

클라우드 인프라 구성

호스팅 : Supabase Cloud Platform

데이터베이스 : PostgreSQL (Supabase Managed)

스토리지 Supabase Storage (음원, 이미지, 녹음 파일 저장)

CDN Vercel (React 프론트엔드 배포)

서버 사양

구성 요소	사양	비고
데이터베이스	PostgreSQL 15+	Supabase 관리형
메모리	2GB+	동시 연결 지원
스토리지	SSD 100GB+	음원 및 데이터 저장
네트워크	1Gbps+	고속 데이터 전송

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

3.4 네트워크 구성 현황

[보안 구성]

SSL/TLS : HTTPS 암호화 통신 (TLS 1.3)

인증 : JWT 기반 토큰 인증 / Supabase Auth 세션 관리

권한 관리 : Row Level Security (RLS)

API 보안 : CORS 정책 적용

[네트워크 프로토콜]

HTTP/HTTPS : REST API 통신 (Express 서버)

WebSocket (WSS) : 실시간 시그널링 (WebRTC SDP/ICE 교환) / 실시간 채팅 메시지

WebRTC (SRTP) : P2P 오디오 스트림 전송

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

3.5 상세 구축 및 구성 내용

3.5.1. 실시간 합주 시스템

[1.1 개요]

WebRTC P2P 기술을 활용한 저지연 실시간 오디오 전송 시스템으로, LAN 환경에서 50-80ms, 같은 도시 내에서 80-120ms의 레이턴시를 달성하여 실시간 온라인 합주를 가능하게 합니다.

[1.2 역할]

- 참여자 간 실시간 오디오 스트림 전송 (서버를 거치지 않는 P2P 직접 연결)
- WebSocket을 통한 시그널링 및 방 관리
- Web Audio API를 통한 개인별 볼륨/패닝 조절
- 연주자/관람자 모드 분리 및 연주 참여 요청 시스템

[1.3 주요 기술]

- **WebRTC**: Peer-to-Peer 오디오 전송
- **Opus 코덱**: 고품질 저지연 오디오 인코딩
- **WebSocket (ws)**: 시그널링 서버
- **Web Audio API**: 실시간 오디오 처리 및 믹싱

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

[1.4 주요 코드]

1.1 SDP 최적화 (저지연 설정)

파일: client/src/contexts/RoomContext.tsx 위치: 15-56줄

```

15 function optimizeSdpForLowLatency(sdp: string): string {
16     let optimizedSdp = sdp
17
18     // Opus 코덱의 fmpo 라인 찾아서 수정
19     // a=fmpo:111 ... 형식
20     optimizedSdp = optimizedSdp.replace(
21         /(a=fmpo:111 .+)/g,
22         (match) => {
23             let newFmpo = match
24             // ptime이 없으면 추가 (10ms 패킷)
25             if (!newFmpo.includes('ptime=')) {
26                 newFmpo += ';ptime=10'
27             }
28             // maxptime이 없으면 추가
29             if (!newFmpo.includes('maxptime=')) {
30                 newFmpo += ';maxptime=10'
31             }
32             // stereo 비활성화
33             if (!newFmpo.includes('stereo=')) {
34                 newFmpo += ';stereo=0'
35             }
36             // FEC 비활성화 (레이턴시 감소)
37             if (!newFmpo.includes('useinbandfec=')) {
38                 newFmpo += ';useinbandfec=0'
39             }
40             // DTX 비활성화 (무음에서도 패킷 전송)
41             if (!newFmpo.includes('usedtx=')) {
42                 newFmpo += ';usedtx=0'
43             }
44             return newFmpo
45         }
46     )
47
48     // a=ptime 라인이 있으면 10으로 수정
49     optimizedSdp = optimizedSdp.replace(/a=ptime:\d+/g, 'a=ptime:10')
50
51     // a=maxptime 라인이 있으면 10으로 수정
52     optimizedSdp = optimizedSdp.replace(/a=maxptime:\d+/g, 'a=maxptime:10')
53
54     console.log('[RTC] SDP optimized for low latency')
55     return optimizedSdp
56 }

```

주요 파라미터 설명:

- ptime=10: 패킷 간격을 10ms로 설정하여 레이턴시 감소
- stereo=0: 모노 전송으로 대역폭 절약
- useinbandfec=0: Forward Error Correction 비활성화로 레이턴시 감소
- usedtx=0: Discontinuous Transmission 비활성화로 무음 구간에서도 패킷 전송

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

1.2 ICE 서버 설정

파일: client/src/contexts/RoomContext.tsx **위치:** 6줄

```
6  const ICE_SERVERS: RTCIceServer[] = [{ urls: 'stun:stun.l.google.com:19302' }]
```

역할: Google STUN 서버를 사용하여 NAT 통과 지원

1.3 개인 믹서 설정

파일: client/src/contexts/RoomContext.tsx **위치:** 62-66줄

```
61  // 개인 믹서 설정
62  export interface MixSettings {
63    volume: number // 0 ~ 1
64    pan: number    // -1 (좌) ~ 1 (우)
65    muted: boolean
66  }
```

역할: 각 참여자의 오디오를 개별적으로 조절하여 개인화된 믹싱 환경 제공

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

1.4 WebSocket 시그널링 서버

파일: server/src/index.ts 위치: 85-103줄

```

84  // 클라이언트 정보 (연주 상태 포함)
85  interface Client {
86      id: string;
87      ws: import('ws').WebSocket;
88      roomId?: string;
89      nickname: string;
90      instrument?: string;
91      isPerforming: boolean;
92      isHost: boolean;
93      userId?: string; // Supabase user ID (중복 연결 방지용)
94  }
95
96  // 참여자 정보 (클라이언트에 전송용)
97  interface Participant {
98      orderId: string;
99      nickname: string;
100     instrument?: string;
101     isPerforming: boolean;
102     isHost: boolean;
103 }

```

역할:

- WebRTC SDP/ICE 시그널링 교환
- 방 참여자 관리 및 동기화
- 연주 요청/승인 처리

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

1.5 Supabase 참여자 수 동기화

파일: server/src/index.ts **위치:** 62-82줄

```

61  // 방 참여자 수 업데이트 함수
62  async function updateRoomParticipants(roomId: string) {
63      if (!supabase) return;
64
65      const room = rooms.get(roomId);
66      const count = room ? room.size : 0;
67
68      try {
69          const { error } = await supabase
70              .from('rooms')
71              .update({ current_participants: count })
72              .eq('id', roomId);
73
74          if (error) {
75              console.error('[DB] Failed to update participants:', error.message);
76          } else {
77              console.log(`[DB] Room ${roomId.slice(0, 8)} participants updated to ${count}`);
78          }
79      } catch (err) {
80          console.error('[DB] Exception updating participants:', err);
81      }
82  }

```

역할: 실시간으로 방의 참여자 수를 데이터베이스에 동기화하여 클라이언트에서 최신 정보 표시

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

3.5.2 AI 음원 분리 시스템

[2.1 개요]

Demucs AI 모델을 사용하여 음원을 4개 트랙(vocals, drums, bass, other)으로 분리하고, 각 트랙을 개별적으로 조절할 수 있는 멀티트랙 플레이어를 제공합니다.

[2.2 역할]

- MP3/WAV 파일 업로드 및 AI 음원 분리
- 분리된 트랙별 개별 재생/음소거
- MR 자동 제작 (보컬 제거)
- 파트별 연습 도구 제공

[2.3 주요 기술]

- **Demucs**: AI 기반 음원 분리 모델
- **Python**: demucs 라이브러리 실행
- **Node.js spawn**: Python 프로세스 실행 및 제어
- **Supabase Storage**: 분리된 트랙 파일 저장

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

[2.4] 주요 코드

1.1 Demucs 음원 분리 서비스

파일: server/src/services/demucsService.ts 위치: 29-117줄

```

26  /**
27   * Python demucs를 사용하여 음원 분리
28   */
29  export async function separateAudioStems(mp3FilePath: string): Promise<SeparationResult> {
30    return new Promise((resolve) => {
31      console.log(`[DEMUCS-Python] 음원 분리 시작: ${mp3FilePath}`)
32
33      const uploadDir = path.dirname(mp3FilePath)
34      const outputDir = path.join(uploadDir, 'separated')
35      const scriptPath = path.join(__dirname, '..', '..', 'demucs_separate.py')
36
37      // 커스텀 Python 스크립트 실행 (soundfile 백엔드 사용)
38      const demucs = spawn('python', [
39        scriptPath,
40        mp3FilePath,
41        outputDir
42      ])
43
44      let stdout = ''
45      let stderr = ''
46
47      demucs.stdout.on('data', (data) => {
48        stdout += data.toString()
49        console.log(`[DEMUCS] ${data}`)
50      })
51
52      demucs.stderr.on('data', (data) => {
53        stderr += data.toString()
54        console.error(`[DEMUCS] ${data}`)
55      })

```

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

```

56
57     demucs.on('close', (code) => {
58         if (code !== 0) {
59             console.error(`[DEMUCS] 프로세스 종료 코드: ${code}`)
60             return resolve({
61                 success: false,
62                 error: `demucs failed with code ${code}: ${stderr}`
63             })
64         }
65
66         try {
67             // Python 스크립트의 JSON 출력 파싱
68             const lines = stdout.trim().split('\n')
69             const jsonLine = lines[lines.length - 1] // 마지막 줄이 JSON
70             const stemsFromScript = JSON.parse(jsonLine)
71
72             // 스템 파일을 업로드 디렉토리로 복사
73             const stems: SeparatedStems = {}
74             const baseName = path.basename(mp3FilePath, path.extname(mp3FilePath))
75
76             for (const [stemType, stemPath] of Object.entries(stemsFromScript)) {
77                 if (fs.existsSync(stemPath as string)) {
78                     const targetFileName = `${baseName}_${stemType}.wav`
79                     const targetPath = path.join(uploadDir, targetFileName)
80
81                     fs.copyFileSync(stemPath as string, targetPath)
82                     stems[stemType as keyof SeparatedStems] = targetPath
83
84                     console.log(`[DEMUCS] 스템 생성: ${stemType} -> ${targetPath}`)
85                 }
86             }
87
88             const availableStems = Object.keys(stems)
89
90             console.log(`[DEMUCS] 음원 분리 완료: ${availableStems.join(', ')}`)
91
92             resolve({
93                 success: true,
94                 stems,
95                 availableStems
96             })
97         } catch (error) {
98             // ...

```

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

```

99     console.error('[DEMUCS] 파일 처리 오류:', error)
100    console.error('[DEMUCS] stdout:', stdout)
101    resolve({
102      success: false,
103      error: error instanceof Error ? error.message : 'Unknown error'
104    })
105  }
106 })

```

역할:

- Python demucs 프로세스를 spawn으로 실행
- stdout에서 JSON 결과 파싱
- 분리된 스템 파일을 업로드 디렉토리에 복사
- 타임아웃 처리 (10분)

2.2 인터페이스 정의

파일: server/src/services/demucsService.ts 위치: 10-24줄

```

10  export interface SeparatedStems {
11    vocals?: string
12    drums?: string
13    bass?: string
14    piano?: string
15    guitar?: string
16    other?: string
17  }
18
19  export interface SeparationResult {
20    success: boolean
21    stems?: SeparatedStems
22    availableStems?: string[]
23    error?: string
24  }

```

역할: 음원 분리 결과 타입 정의

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

[3.4.3. 사용자 인증 시스템]

[3.1 개요]

Supabase Auth를 활용한 이메일/비밀번호 기반 인증 시스템으로, JWT 토큰을 사용한 세션 관리와 프로필 정보 저장을 제공합니다.

[3.2 역할]

- 회원가입/로그인/로그아웃 처리
- JWT 토큰 기반 세션 관리
- 사용자 프로필 관리
- 인증 상태 전역 관리

[3.3 주요 기술]

- Supabase Auth: 이메일/비밀번호 인증
- JWT: 세션 토큰 관리
- React Context API: 인증 상태 전역 관리
- PostgreSQL: 사용자 프로필 저장

[3.4 주요 코드]

1. AuthContext 정의

파일: client/src/context/AuthContext.tsx 위치: 17-26줄

```

17  type AuthContextType = {
18      user: User | null
19      loading: boolean
20      alert: AuthAlert | null
21      setAlert: (alert: AuthAlert | null) => void
22      signUp: (data: SignUpData) => Promise<void>
23      signIn: (email: string, password: string) => Promise<void>
24      signOut: () => Promise<void>
25      isSupabaseReady: boolean
26  }

```

역할: 인증 관련 모든 상태와 메서드를 제공하는 Context 타입

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

2. 회원가입 처리

파일: client/src/contexts/AuthContext.tsx 위치: 63-126줄

```

63   const signUp = async (data: SignUpData) => {
64     if (!supabase) {
65       showAlert({ type: 'error', message: 'Supabase 환경변수를 설정한 뒤 다시 시도하세요.' })
66       throw new Error('Supabase not configured')
67     }
68
69     setLoading(true)
70     showAlert(null)
71
72     try {
73       // 1. 사용자 인증 생성 (metadata에 프로필 정보 포함)
74       const { data: authData, error: authError } = await supabase.auth.signUp({
75         email: data.email.trim(),
76         password: data.password,
77         options: {
78           data: {
79             name: data.name.trim(),
80             nickname: data.nickname.trim(),
81             birth_date: data.birthDate,
82             gender: data.gender,
83             phone: data.phone?.trim() || null,
84           },
85         },
86       })
87
88       if (authError) throw authError
89       if (!authData.user) throw new Error('회원가입에 실패했습니다.')
90
91       // 2. 인증 완료될 때까지 대기 (최대 2초)
92       await new Promise((resolve) => setTimeout(resolve, 2000))
93
94       // 3. 프로필 정보 저장
95       const { error: profileError } = await supabase.from('profiles').insert({
96         id: authData.user.id,
97         email: data.email.trim(),
98         name: data.name.trim(),
99         nickname: data.nickname.trim(),
100        birth_date: data.birthDate,
101        gender: data.gender,
102        phone: data.phone?.trim() || null,
103      })

```


프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

```

105     if (profileError) {
106         console.error('Profile creation error:', profileError)
107         console.error('Error details:', JSON.stringify(profileError, null, 2))
108         // 프로필 생성 실패 시에도 회원가입은 완료된 것으로 처리
109         showAlert({
110             type: 'info',
111             message: '회원가입은 완료되었습니다. 이메일 확인 후 로그인해주세요.',
112         })
113     } else {
114         showAlert({ type: 'success', message: '회원가입이 완료되었습니다! 로그인해주세요.' })
115     }
116 } catch (error) {
117     console.error('SignUp error:', error)
118     showAlert({
119         type: 'error',
120         message: error instanceof Error ? error.message : '회원가입 중 오류가 발생했습니다.',
121     })
122     throw error
123 } finally {
124     setLoading(false)
125 }
126 }

```

역할:

1. Supabase Auth에 사용자 인증 정보 생성
2. 프로필 테이블에 사용자 정보 저장
3. 에러 처리 및 사용자 피드백 제공

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

3. 로그인 처리

파일: client/src/contexts/AuthContext.tsx **위치:** 128-153줄

```

128   const signIn = async (email: string, password: string) => {
129     if (!supabase) {
130       showAlert({ type: 'error', message: 'Supabase 환경변수를 설정한 뒤 다시 시도하세요.' })
131       throw new Error('Supabase not configured')
132     }
133
134     setLoading(true)
135     showAlert(null)
136
137     try {
138       const { error } = await supabase.auth.signInWithPassword({
139         email: email.trim(),
140         password,
141       })
142       if (error) throw error
143       showAlert({ type: 'success', message: '로그인 성공' })
144     } catch (error) {
145       showAlert({
146         type: 'error',
147         message: error instanceof Error ? error.message : '로그인 중 오류가 발생했습니다.',
148       })
149       throw error
150     } finally {
151       setLoading(false)
152     }
153   }

```

역할: 이메일/비밀번호로 로그인 처리 및 JWT 토큰 발급

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

4. 커뮤니티 시스템

[4.1 개요]

5가지 카테고리 게시판(자유, 합주모집, 악기리뷰, 팁공유, 공연정보)과 실시간 댓글 시스템을 제공하는 음악가 전용 커뮤니티입니다.

[4.2 역할]

- 게시물 CRUD (생성, 읽기, 수정, 삭제)
- 실시간 댓글 시스템 (Supabase Realtime)
- 좋아요 기능
- 악기별 필터링
- 트렌딩 태그

[4.3 주요 기술]

- **Supabase Realtime**: 실시간 데이터 구독
- **PostgreSQL**: 게시물, 댓글 저장
- **Row Level Security (RLS)**: 데이터 접근 제어
- **React**: UI 컴포넌트

[4.4 주요 코드]

1. 게시물 조회

파일: client/src/pages/Community.tsx 위치: 38-52줄

```

37 // Fetch posts from database
38 const fetchPosts = useCallback(async () => {
39   try {
40     setLoading(true)
41     const data = await communityApi.getPosts({
42       instrument: selectedInstrument,
43       tag: selectedTag ?? undefined,
44       userId: user?.id
45     })
46     setPosts(data)
47   } catch (error) {
48     console.error('Failed to fetch posts:', error)
49   } finally {
50     setLoading(false)
51   }
52 }, [selectedInstrument, selectedTag, user?.id])

```

역할: 악기 필터 및 태그를 적용하여 게시물 조회

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

2. 좋아요 토글

파일: client/src/pages/Community.tsx 위치: 80-108줄

```

80     const handleLike = async (postId: string, e?: React.MouseEvent) => {
81         e?.stopPropagation()
82
83         if (!user) {
84             alert('좋아요를 누르려면 로그인が必要です.')
85             return
86         }
87
88         try {
89             const { liked, likesCount } = await communityApi.toggleLike(postId, user.id)
90
91             // Update local state
92             setPosts(prev => prev.map(post => {
93                 if (post.id === postId) {
94                     return {
95                         ...post,
96                         likes_count: likesCount,
97                         liked_by_user: liked
98                     }
99                 }
100                 return post
101             }))
102         } catch (error) {
103             console.error('Failed to toggle like:', error)
104         }
105     }
106
107     const handleClick = (postId: string) => {
108         navigate(`/community/${postId}`)
109     }

```

역할:

- 좋아요 토글 처리
- 로컬 상태 업데이트로 즉각적인 UI 반영

3. 인기 게시물 필터링

파일: client/src/pages/Community.tsx 위치: 73-78줄

```

72     // 인기 게시물 (좋아요 10개 이상, 좋아요 많은 순)
73     const popularPosts = useMemo(() => {
74         return [...posts]
75             .filter(post => post.likes_count >= 10)
76             .sort((a, b) => b.likes_count - a.likes_count)
77             .slice(0, 3)
78     }, [posts])

```

역할: 좋아요 10개 이상인 게시물을 좋아요 순으로 정렬하여 상위 3개 표시

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

5. 결제 시스템

[5.1 개요]

토스페이먼츠 SDK를 연동한 안전한 결제 시스템으로, 3단계 Freemium 요금제(Free, Standard, Pro)를 제공합니다. (요금제 별 기능 차별화는 구현 예정)

[5.2 역할]

- 토스페이먼츠 결제 연동
- 결제 승인, 취소, 조회
- 구독 플랜 관리
- 주문 ID 생성 및 검증

[5.3 주요 기술]

- 토스페이먼츠 SDK: 결제 위젯 및 API
- Express: 결제 API 서버
- Supabase: 구독 정보 저장
- PCI-DSS: 결제 보안 표준 준수

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

[5.4 주요 코드]

1. 결제 승인

파일: server/src/services/paymentService.ts 위치: 15-44줄

```

14  // 결제 승인 (토스페이먼츠)
15  async confirmPayment(paymentKey: string, orderId: string, amount: number): Promise<PaymentResponse> {
16      try {
17          const response = await axios.post(
18              `${TOSS_PAYMENTS_BASE_URL}/payments/confirm`,
19              {
20                  paymentKey,
21                  orderId,
22                  amount
23              },
24              {
25                  headers: {
26                      'Authorization': this.getAuthHeader(),
27                      'Content-Type': 'application/json'
28                  }
29              }
30          )
31
32          return {
33              success: true,
34              paymentKey: response.data.paymentKey,
35              orderId: response.data.orderId
36          }
37      } catch (error: any) {
38          console.error('Payment confirmation failed:', error.response?.data || error.message)
39          return {
40              success: false,
41              error: error.response?.data?.message || '결제 승인에 실패했습니다.'
42          }
43      }
44  }

```

역할:

- 토스페이먼츠 API로 결제 승인 요청
- Basic 인증 헤더 사용
- 에러 처리

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

2. 결제 요청 검증

파일: server/src/services/paymentService.ts 위치: 95-108줄

```

95  validatePaymentRequest(request: PaymentRequest): { valid: boolean; error?: string } {
96      const { planType, amount } = request
97
98      if (!['standard', 'pro'].includes(planType)) {
99          return { valid: false, error: '유효하지 않은 플랜입니다.' }
100     }
101
102     const expectedAmount = PLAN_PRICES[planType]
103     if (amount !== expectedAmount) {
104         return { valid: false, error: '결제 금액이 올바르지 않습니다.' }
105     }
106
107     return { valid: true }
108 }

```

역할:

- 플랜 타입 및 결제 금액 검증
- 보안: 클라이언트에서 전송된 금액이 실제 플랜 가격과 일치하는지 확인

3. 주문 ID 생성

파일: server/src/services/paymentService.ts 위치: 111-128줄

```

110  // 주문 ID 생성
111  generateOrderId(userId: string, planType: string): string {
112      // userId를 안전한 형태로 변환 (영문, 숫자만)
113      const safeUserId = userId.replace(/^[a-zA-Z0-9]/g, '').substring(0, 8) || 'user'
114      const timestamp = Date.now().toString().slice(-8) // 마지막 8자리만
115      const random = Math.random().toString(36).substring(2, 6) // 4자리 랜덤
116
117      // orderId 생성 (영문, 숫자, -, _ 만 사용, 6-64자)
118      const orderId = `order-${safeUserId}-${planType}-${timestamp}-${random}`
119
120      // 길이 검증 (6-64자)
121      if (orderId.length < 6 || orderId.length > 64) {
122          // 길이가 맞지 않으면 더 간단한 형태로 생성
123          return `order-${timestamp}-${random}`
124      }
125
126      return orderId
127  }

```

역할:

- 유니크한 주문 ID 생성
- 토스페이먼츠 규격 준수 (6-64자, 영문/숫자/하이픈/언더스코어만 사용)

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

6. 오디오 설정 관리

[6.1 개요]

사용자의 오디오 입출력 장치를 관리하고, 악기별 최적화된 프리셋을 제공하는 오디오 설정 시스템입니다.

[6.2 역할]

- 오디오 입출력 장치 선택
- 샘플레이트, 채널 수 설정
- 에코 제거, 노이즈 억제, 자동 게인 제어 설정
- 악기별 프리셋 제공 (보컬, 기타, 베이스, 키보드, 드럼)
- 입력 레벨 미터 시각화

[6.3 주요 기술]

- **MediaDevices API**: 오디오 장치 목록 조회
- **getUserMedia API**: 마이크 권한 요청 및 스트림 생성
- **Web Audio API**: 입력 레벨 분석
- **React Context API**: 오디오 설정 전역 관리

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

[6.4 주요 코드]

1. 오디오 설정 인터페이스

파일: client/src/contexts/AudioSettingsContext.tsx 위치: 7-16줄

```

6  // 오디오 설정 타입
7  export interface AudioSettings {
8      inputDeviceId: string | null
9      outputDeviceId: string | null
10     sampleRate: number
11     channelCount: number
12     echoCancellation: boolean
13     noiseSuppression: boolean
14     autoGainControl: boolean
15     stereoMode: StereoMode // 스테레오 입력 시 출력 모드
16 }

```

역할: 오디오 설정 옵션 타입 정의

2. 기본 설정

파일: client/src/contexts/AudioSettingsContext.tsx 위치: 78-87줄

```

78 const defaultSettings: AudioSettings = {
79     inputDeviceId: null,
80     outputDeviceId: null,
81     sampleRate: 48000,
82     channelCount: 1,
83     echoCancellation: false, // 악기용이므로 기본 OFF
84     noiseSuppression: false,
85     autoGainControl: false,
86     stereoMode: 'mono', // 기본값: 스테레오 입력을 모노로 합침
87 }

```

역할:

- 악기 연주에 최적화된 기본 설정
- 에코 제거, 노이즈 억제, 자동 게인 제어를 모두 비활성화하여 원음 보존

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

3. 악기별 프리셋

파일: client/src/contexts/AudioSettingsContext.tsx **위치:** 90-125줄

```

89  // 프리셋 설정
90  const presets: Record<AudioPreset, Partial<AudioSettings>> = {
91    vocal: {
92      channelCount: 1,
93      echoCancellation: true,
94      noiseSuppression: true,
95      autoGainControl: true,
96    },
97    guitar: {
98      channelCount: 1,
99      echoCancellation: false,
100     noiseSuppression: false,
101     autoGainControl: false,
102   },
103   bass: {
104     channelCount: 1,
105     echoCancellation: false,
106     noiseSuppression: false,
107     autoGainControl: false,
108   },
109   keyboard: {
110     channelCount: 2,
111     echoCancellation: false,
112     noiseSuppression: false,
113     autoGainControl: false,
114   },
115   drums: {
116     channelCount: 2,
117     echoCancellation: false,
118     noiseSuppression: false,
119     autoGainControl: false,
120   },
121   custom: {},
122 }

```

역할:

- 악기별로 최적화된 오디오 설정 제공
- 보컬: 에코/노이즈 제거, 자동 게인 활성화
- 악기 : 원음 보존을 위해 모든 처리 비활성화

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

3.6 구축 결과

3.6.1 개발 통계

총 개발 파일:

- React 컴포넌트: 30개 이상
- TypeScript 파일: 50개 이상
- 총 코드 라인 수: 15,000줄 이상

API 엔드포인트:

- /api/music/separate: 음원 분리
- /api/payment/confirm: 결제 승인
- /api/payment/cancel: 결제 취소
- WebSocket 시그널링: 10개 이상의 메시지 타입

데이터베이스 테이블:

- profiles, rooms, posts, comments
- recordings, subscriptions, payments
- messages, notifications, room_participants

8.2 성능 지표

- 오디오 레이턴시: LAN 환경 50-80ms, 같은 도시 80-120ms
- 페이지 로딩 속도: 평균 2초 이내
- API 응답 시간: 평균 500ms 이내
- 동시 접속: 방당 최대 8명 지원
- 음원 분리 시간: 3분 곡 기준 3-5분

8.3 보안 구현

- 인증: JWT 토큰 기반 세션 관리
- 데이터베이스: Row Level Security (RLS) 적용
- 통신: HTTPS/WSS 암호화
- 결제: 토스페이먼츠 PCI-DSS 준수
- 환경변수: 클라이언트/서버 키 분리

8.4 특징

저지연 최적화:

- Opus ptime 10ms 설정
- FEC/DTX 비활성화
- P2P 직접 연결

사용자 경험:

- 실시간 참여자 수 동기화
- 개인화된 믹싱 환경
- 악기별 최적화 프리셋
- 직관적인 UI/UX

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

확장성:

- Monorepo 구조로 통합 관리
- Context API로 관심사 분리
- TypeScript로 타입 안정성 확보
- Supabase BaaS로 인프라 관리 간소화

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

4. 개별 후기

- 홍채원 후기

- 신지웅 후기

프로젝트 완료 보고서		
프로젝트 주제	웹 기반 실시간 합주 및 음악 협업 종합 플랫폼	
단계 : 프로젝트 완료	작성자 : 홍채원	작성일 : 2026.01.28

● 전유진 후기